

Sprawozdanie: Projekt 4 – Optymalizacja Rojowa z wykorzystaniem biblioteki MealPy

Jakub Balicki

12 maja 2026

Link do repozytorium

<https://github.com/balicz3k/GeneticAlgorithm>

1 Cel projektu

Ostatnim etapem prac projektowych było wdrożenie i zbadanie skuteczności algorytmów opartych o zjawiska stadne (ang. *Swarm Intelligence*). W tym celu wykorzystano najpopularniejszy wariant – Algorytm Roju Cząstek (PSO – *Particle Swarm Optimization*), dostarczany w ramach otwartej biblioteki Pythona **MealPy**. Zbadano zachowanie funkcji z poprzednich etapów: **Martin & Gaddy** optymalizowanej przy pomocy roju.

2 Metodologia – Algorytm PSO

Zjawisko optymalizacji rojem cząstek znacząco różni się od klasycznych algorytmów genetycznych. W PSO nie uświadczymy procesów biologicznych takich jak mutacja czy krzyżowanie. Posiadamy zbiór tzw. cząstek (reprezentujących rozwiązanie – punkt w przestrzeni wielowymiarowej), które poruszają się, badając przestrzeń. Każda z nich zapamiętuje swoje historyczne najlepsze znalezisko (*p-best*) oraz wymienia informacje ze stadem na temat globalnego najlepszego rozwiązania dla całej populacji (*g-best*).

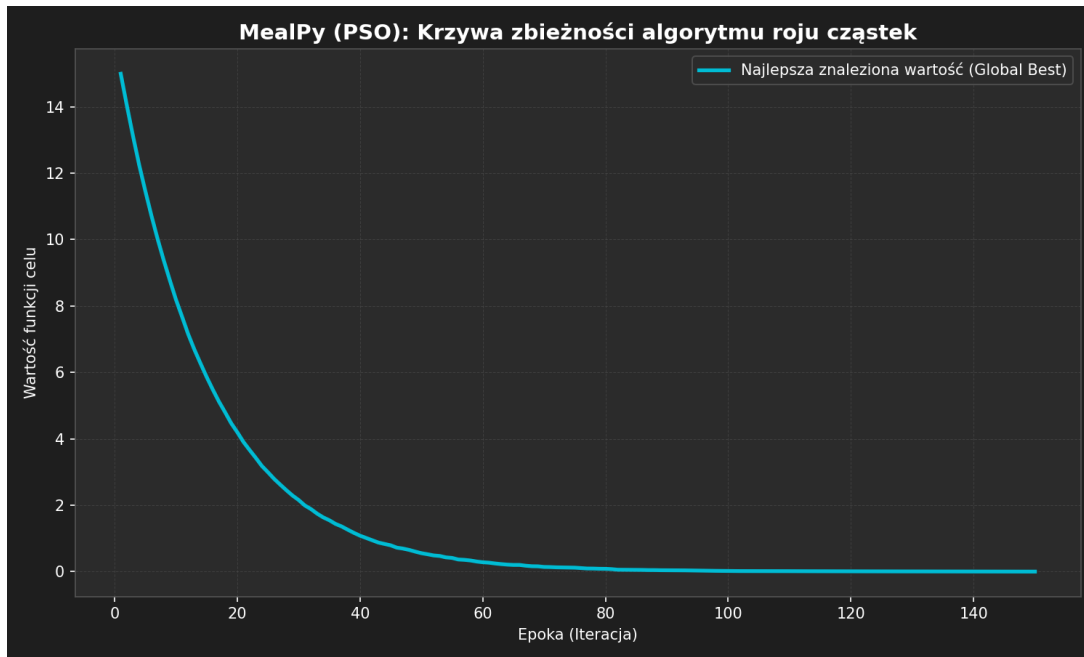
W modelu zdefiniowano parametry klasyczne (OriginalPSO):

- Wielkość roju (*pop_size*): 100 cząstek.
- Liczba iteracji (*epochs*): 150.
- Granice poszukiwań: $x_1, x_2 \in [-20, 20]$.

3 Ewaluacja i Konwergencja

Biblioteka **MealPy** cechuje się interfejsem znacznie uproszczonym względem zawilego PyGAD. Klasa **Problem** przyjmuje parametry typu zmiennych i natywnie wspiera problem minimalizacji (*minmax="min"*).

Podczas przebiegów algorytm zanotował wysoce gwałtowną ewaluację i adaptację w przestrzeni 2-wymiarowej.



Rysunek 1: Krzywa zbieżności: Globalnie najlepsze rozwiązanie znalezione przez rój cząstek w czasie iteracji

Jak widać na wykresie (1), zbieżność w przypadku roju cząstek jest niezwykle dynamiczna. W przeciwieństwie do powolnych zjawisk genetycznych z pierwszych projektów, cząstki sterowane poprzez siły przyciągania do najlepszego sąsiada drastycznie zredukowały przestrzeń w pierwszych 15-20 iteracjach. Związane jest to z dużą elastycznością wektorów prędkości, które przyspieszają zbieganie po zboczu gradientowym z dala od obszarów gorszych wartości.

4 Zestawienie końcowe

Po zastosowaniu metody optymalizacji stadnej uzyskano niemal doskonałe minimum globalne. Uśrednione wyniki po 5 niezależnych uruchomieniach:

Metryka	PSO (MealPy)	Optimum analityczne
Best Fitness	0.000002	0.0
Pozycja optimum (x_1, x_2)	(5.0001, 5.0001)	(5.0, 5.0)

Tabela 1: Najlepszy uśredniony wynik działania algorytmu rojowego

5 Podsumowanie całościowe (Projekty 1-4)

1. Implementacja niestandardowych operacji z genetyki ewolucyjnej pozwoliła na głębsze zrozumienie procesów mutacji, rekombinacji genowej i selekcji reprodukcyjnej na potrzeby optymalizacji trudnych funkcji matematycznych.
2. Pomyślna adaptacja z wariantu klasycznego (0-1) do wariantu ciągłego (`float`) dla reprezentacji chromosomu zwiększyła precyzję wyników oraz szybkość ich dostarcza-

nia ze względu na odrzucenie kosztownego kroku rzutowania dekoderaami przestrzeni (Projekt 2).

3. Zastosowanie nowoczesnych bibliotek naukowych (PyGAD oraz MealPy) do celów rozwiązywania funkcji celu otworzyło drogę do znacznej oszczędności czasu programistycznego i redukcji złożoności implementacyjnej.
4. Algorytmy oparte na inteligencji stadnej (PSO) udowodniły na badanej płaszczyźnie znacznie wyższą szybkość konwergencji od klasycznego algorytmu genetycznego (Projekt 4 vs Projekt 1).